

Consistent Hashing Ring

Cluster of LRU caches with two placement algorithms: consistent hashing (ring + virtual nodes) & modulo hashing

Author: Thanh Trung Le, Track: B — Implementation and Building Something

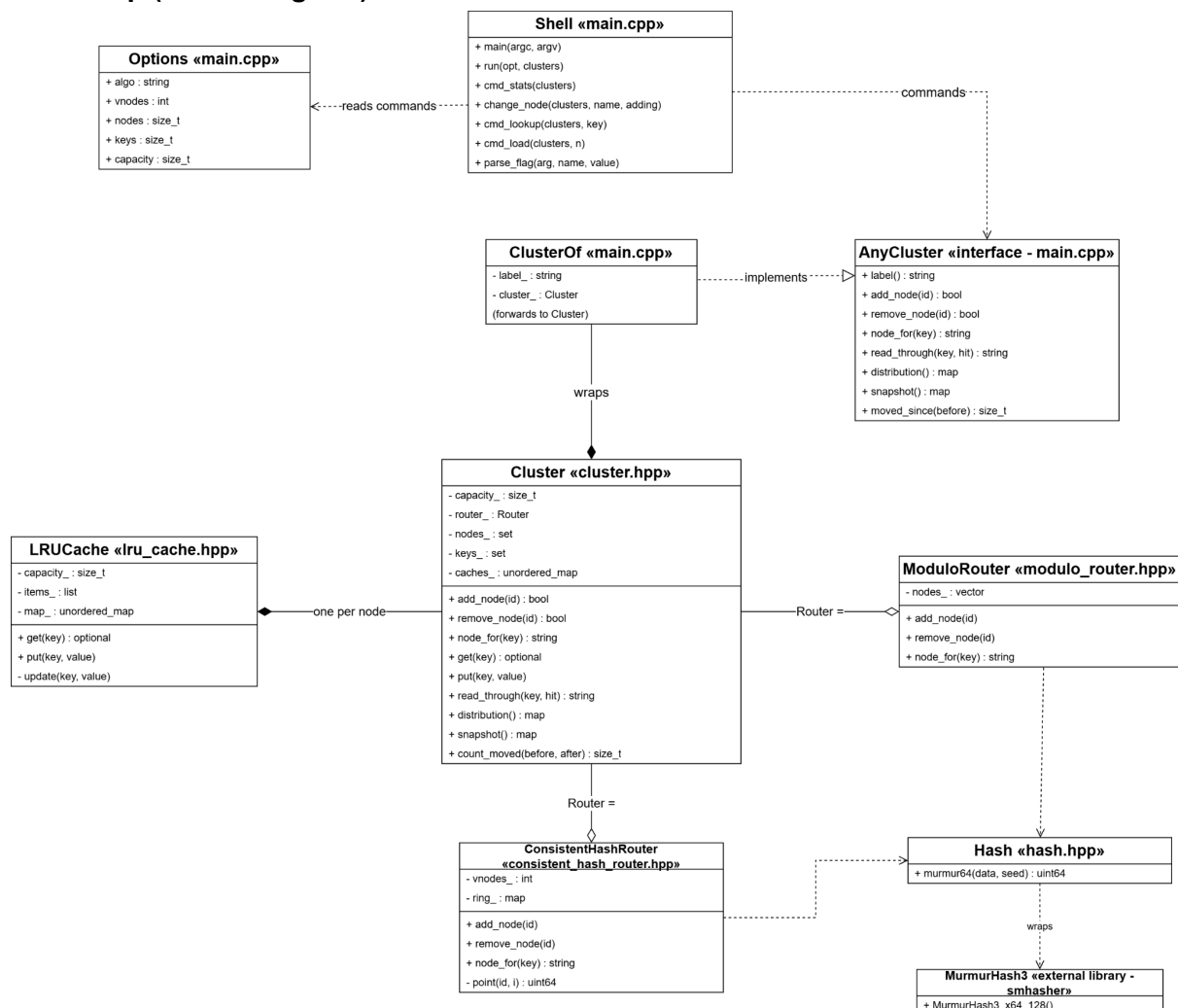
Repository: [Louis-Le06/Consistent-hashing-ring](https://github.com/Louis-Le06/Consistent-hashing-ring)

Video walkthrough: https://youtu.be/chlAmTN_39c

Overview

Consistent hashing is a data distribution technique designed to minimise the amount of relocated data when the number of servers changes. Specifically, the Modulo hashing technique uses the formula 'Server_Index = hash_function(key) % N' (N is the number of servers) to distribute data. This method is highly effective when the number of servers remains constant. However, adding or removing a server requires recalculating the locations of almost all the keys, leading to cache misses or system overload. Hence, consistent hashing was developed to address this drawback, as it ensures that only a fraction of the keys ($\sim 1/N$, where N is the number of servers) need to be moved.

Code map (Class Diagram)



Track B

Functional description: add/remove node, key lookup, distribution statistics, measure number of moved keys

The algorithms workflow

add-node A : main.cpp → Cluster::add_node → ConsistentHashRouter::add_node
→ murmur64("A#0" ... "A#149") → 150 points on ring
lookup k : main.cpp → Cluster::node_for → ConsistentHashRouter::node_for
→ murmur64("k") → lower_bound on ring → node

Builds

```
g++ -std=c++17 -O2 -I. -Ismhasher/src main.cpp smhasher/src/MurmurHash3.cpp -o demo  
./demo
```

CLI commands

Commands	Function
add-node <name>	Add a node
remove-node <name>	Remove a node
lookup <key>	Which node owns this key, if key doesn't exist → add key
stats	Key distribution per node
load <n>	Generate n sample keys
nodes	List the nodes
help	This list
quit	Exit

Startup flags

Flags	Default
--algo=both ring modulo	both
--virtual-nodes=N	150
--nodes=N	0
--keys=N	0
--capacity=N	100000

I chose CLI because it is easier to measure how keys and nodes are distributed instantly. It also makes writing output tests easier since everything is just texts. However, GUI is recommended for visualisation. For instance, while modulo hashing helps the server achieve load balancing, when adding or removing servers necessitates the redistribution of almost all keys, leading to cache miss increase, is more illustrated compared to CLI.

Workout example

```
> g++ -std=c++17 -O2 -I. -Ismhasher/src main.cpp smhasher/src/MurmurHash3.cpp -o demo
```

```
> ./demo --virtual-nodes=100 --nodes=4 --keys=100000
```

```
→ [ ringcache - algo=both virtual-nodes=100 capacity=100000 per node  
    started with 4 nodes
```

```
    loaded 100000 keys, each cluster now knows 100000 keys ]
```

```
> stats
```

```
→ [ ring
```

```
node-0 #####.. 24,800 0.992x
```

```
node-1 #####. 25,120 1.005x
```

```
node-2 ##### 25,851 1.034x
```

```
node-3 #####.. 24,229 0.969x
```

```
4 nodes, 100,000 keys, max/mean 1.034x
```

```
modulo
```

```
node-0 #####. 24,830 0.993x
```

```
node-1 #####. 24,977 0.999x
```

```
node-2 ##### 25,144 1.006x
```

```
node-3 #####. 25,049 1.002x
```

```
4 nodes, 100,000 keys, max/mean 1.006x ]
```

```
> add-node node-4
```

```
→ [ added node-4 (5 nodes)
```

```
ring    keys moved: 21299/100000 (21.3%)
```

```
modulo keys moved: 79975/100000 (80.0%)
```

```
ideal for consistent hashing: about 20.0% ]
```

```
> stats
```

```
→ [ ring
```

```
node-0 #####.... 19,253 0.963x
```

```
node-1 #####.... 19,586 0.979x
```

```
node-2 ##### 22,188 1.109x
```

```
node-3 #####..... 17,674 0.884x
```

```
node-4 #####.. 21,299 1.065x
```

```
5 nodes, 100,000 keys, max/mean 1.109x
```

```
modulo
```

```
node-0 #####. 19,736 0.987x
```

```
node-1 #####. 20,031 1.002x
```

```
node-2 #####. 20,035 1.002x
```

```
node-3 ##### 20,146 1.007x
```

```
node-4 #####. 20,052 1.003x
```

```
5 nodes, 100,000 keys, max/mean 1.007x ]
```

```

> quit
> g++ -std=c++17 -O2 -I. -Ismhasher/src main.cpp smhasher/src/MurmurHash3.cpp -o demo
> ./demo --virtual-nodes=100 --nodes=4 --keys=100000
→ [ ringcache - algo=both virtual-nodes=100 capacity=100000 per node
    started with 4 nodes
    loaded 100000 keys, each cluster now knows 100000 keys ]
> nodes
→ [ node-0 node-1 node-2 node-3 ]

> remove-node node-3
→ [ removed node-3 (3 nodes)
    ring   keys moved: 24229/100000 (24.2%)
    modulo keys moved: 75137/100000 (75.1%)
    ideal for consistent hashing: about 25.0% ]

> stats
→[
  ring
  node-0 ##### 35,761 1.073x
  node-1 #####.... 31,098 0.933x
  node-2 #####... 33,141 0.994x
  3 nodes, 100,000 keys, max/mean 1.073x

  modulo
  node-0 ##### 33,480 1.004x
  node-1 #####. 33,365 1.001x
  node-2 #####. 33,155 0.995x
  3 nodes, 100,000 keys, max/mean 1.004x
]

```

Results: number of moved keys when adding or removing a node for consistent hashing would be $\sim 1/N$

Number of nodes (100 virtual nodes)	Reallocated keys (Modulo hashing)	Reallocated keys (Consistent hashing)
4 → 5	79975/100000 (80.0%)	21299/100000 (21.3%)
4 → 3	75137/100000 (75.1%)	24229/100000 (24.2%)

What I learned

1. At first I thought that I could code a hash function on my own if I understood the algorithm mechanic, but the more I do, the more I realise how difficult it is to implement. So instead, I was looking for an external library on Github. After searching carefully, some candidates repo appeared such as xxHash and MurmurHash3. xxHash is considered faster than MurmurHash3 when it comes to hash a large amount of keys, but since the project was simple and short, MurmurHash3 is recommended.
2. I realised that when the number of servers is fixed, modulo hashing actually achieves better load balancing than consistent hashing. Accordingly, with 3,4, or 5 nodes, the max/mean ratio (calculated by comparing the load on each node against a perfectly equal distribution, taking the highest value to represent the load balance level) for modulo hashing is lower than consistent hashing.
3. Initially, I used an 'std::unordered_map' to store positions and nodes for the hashing, but then the program sent errors during testing. After thinking about the errors, I realised that when looking up a new key, the key effectively moves clockwise to find the nearest node for assignment according to Alex Xu in System Design Interview book. Specifically, the key is hashed into a numeric value, and the system searches for the nearest node in a clockwise direction based on that value. Consequently, the hash ring itself needs to be sorted beforehand to support this operation. Therefore, I switched to using 'std::map' to store the positions and nodes, and after testing and fixing bugs with Claude's assistance, the program ran correctly.

Tool used and how much

- I used Claude to integrate the MurmurHash3 library and write main.cpp, cluster.hpp, hash.hpp. I wrote consistent_hash_router.hpp, modulo_router.hpp, and lru_cache.hpp on my own. Moreover, I also used Claude to debug and add extra commands to the program for better benchmark testing.